Practical 3: Source Code and Output

Code :

```
# Tower of Hanoi using BFS (Breadth First Search)
# Disks: 1 (smallest), 2, 3 (largest)

from collections import deque

# Initial state:
# Rod A has all disks, Rod B and Rod C are empty
start = ((3, 2, 1), (), ())

# Goal state:
# All disks should be on Rod C
goal = ((), (), (3, 2, 1))

# Queue for BFS
queue = deque()

# Each queue item stores:
# (current_state, path_of_moves)
queue.append((start, []))

# Set to remember visited states
visited = set()
visited.add(start)

# Function to generate all valid next states
def get_next_states(state):

    next_states = []

    # Convert tuples to lists so we can modify them
    rods = [list(rod) for rod in state]

    # Try moving the top disk from every rod
    for i in range(3):  # source rod

        if not rods[i]:
            continue  # rod is empty

        disk = rods[i][-1]  # top disk

        for j in range(3):  # destination rod

            if i == j:
                continue  # cannot move to same rod

            # Valid move:
            # destination empty OR top disk there is larger
            if not rods[j] or rods[j][-1] > disk:

                # Make a copy of rods
                new_rods = [r[:] for r in rods]

                # Remove disk from source
                moved_disk = new_rods[i].pop()
```

```
                # Put disk on destination
                new_rods[j].append(moved_disk)

                # Convert lists back to tuples so state can be stored in a set
                new_state = tuple(tuple(r) for r in new_rods)

                # Save state and move description
                move_text = f"Move disk {disk} from Rod {i+1} to Rod {j+1}"
                next_states.append((new_state, move_text))

    return next_states


# BFS algorithm
while queue:

    # Take the first state from queue
    state, path = queue.popleft()

    # If goal reached, print solution
    if state == goal:

        print("Solution found using BFS:\n")

        for step in path:
            print(step)

        print("\nFinal State:")
        print(state)
        break

    # Generate all possible next states
    for next_state, move in get_next_states(state):

        # Process only unvisited states
        if next_state not in visited:

            visited.add(next_state)

            # Add next state to queue with updated path
            queue.append((next_state, path + [move]))
```

Output:

Move disk 1 from Rod 1 to Rod 3

Move disk 2 from Rod 1 to Rod 2

Move disk 1 from Rod 3 to Rod 2

Move disk 3 from Rod 1 to Rod 3

Move disk 1 from Rod 2 to Rod 1

Move disk 2 from Rod 2 to Rod 3

Move disk 1 from Rod 1 to Rod 3

Final State:

((), (), (3, 2, 1))